

MECHATRONICS AND IoT

ASSIGNMENT REPORT – 1

TITLE:

Design and Develop an Industrial Diesel Generator Monitoring and Maintenance System. The system monitors generator performance parameters and supports preventive maintenance operations.

TEAM MEMEBERS:

HARISANKARAN S (24MER013)

MUTHUKUMAR M (24MER034)

MONISH C (24MER032)

Project Overview:

Industrial Diesel Generators (DGs) are widely used as backup and continuous power sources in industries, hospitals, commercial buildings, and manufacturing plants. Unexpected generator failures can interrupt production, damage equipment, and increase maintenance costs.

The proposed Industrial Diesel Generator Monitoring and Maintenance System continuously monitors important operating parameters such as engine temperature, oil pressure, fuel level, battery voltage, engine RPM, and vibration. An ESP32 microcontroller processes the sensor data and provides real-time monitoring through an LCD display and cloud-based IoT dashboard.

The system also supports preventive maintenance by generating alerts whenever operating conditions exceed safe limits or when scheduled maintenance is due. This reduces downtime, improves equipment reliability, and extends generator life.

Problem Statement:

Industries rely on diesel generators during power failures. However, conventional DG maintenance is generally performed at fixed intervals rather than based on the actual condition of the generator.

Major problems include:

- Unexpected engine overheating.
- Low fuel causing sudden shutdown.
- Battery failure during startup.
- Low oil pressure damaging the engine.
- Lack of remote monitoring in industries.
- High maintenance and downtime costs.

Therefore, a low-cost intelligent monitoring system is required for continuous condition monitoring and preventive maintenance.

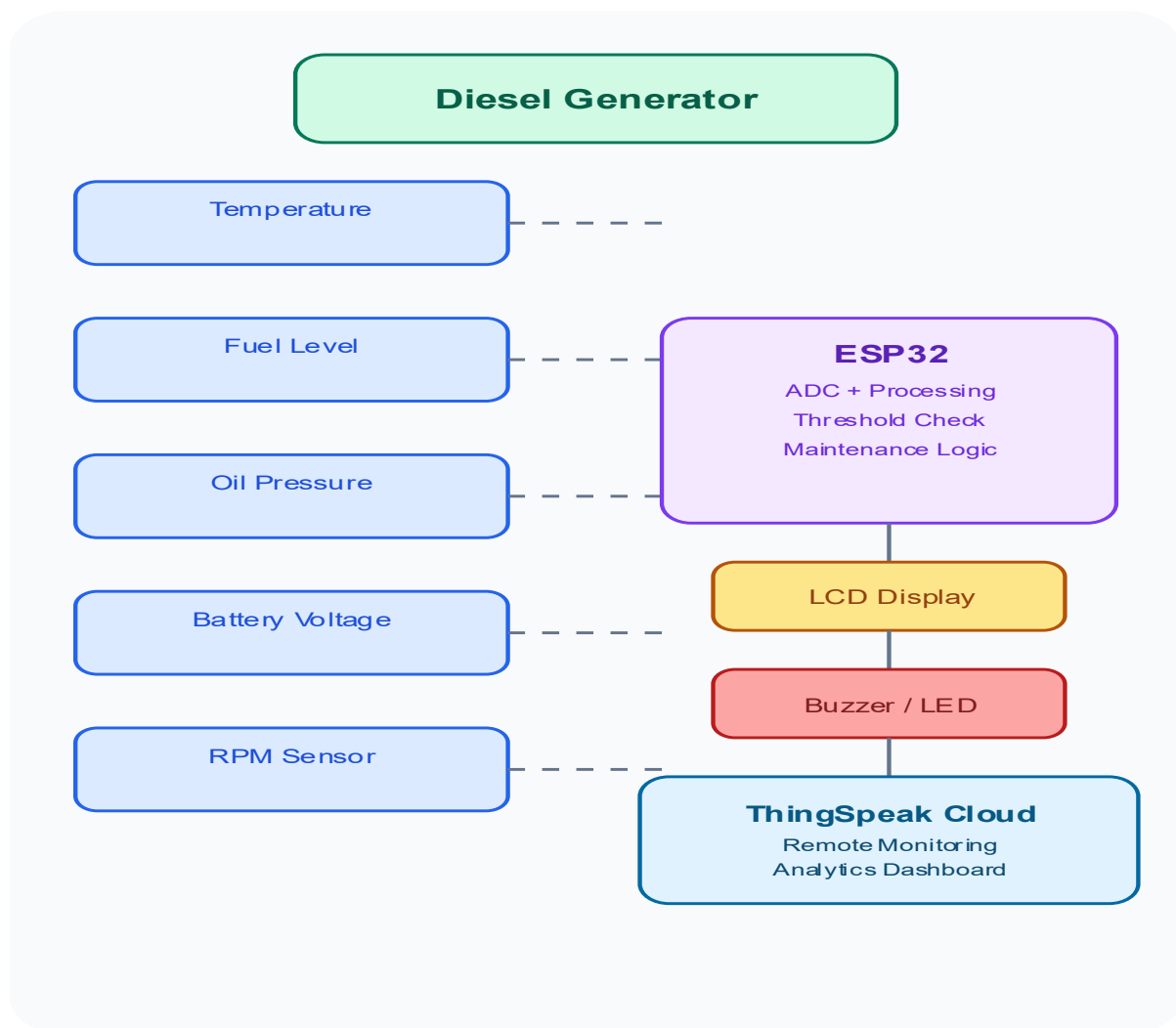
Proposed Solution:

The proposed system uses multiple sensors connected to an ESP32 controller.

Working Principle

1. Sensors continuously measure generator parameters.
2. ESP32 reads all sensor values.
3. Data is compared with preset safety limits.
4. Generator status is classified as Normal, Warning, or Critical.
5. LCD displays live readings.
6. Buzzer activates during dangerous conditions.
7. ESP32 uploads data to ThingSpeak using Wi-Fi.
8. Maintenance alerts are generated when operating hours exceed the service limit.

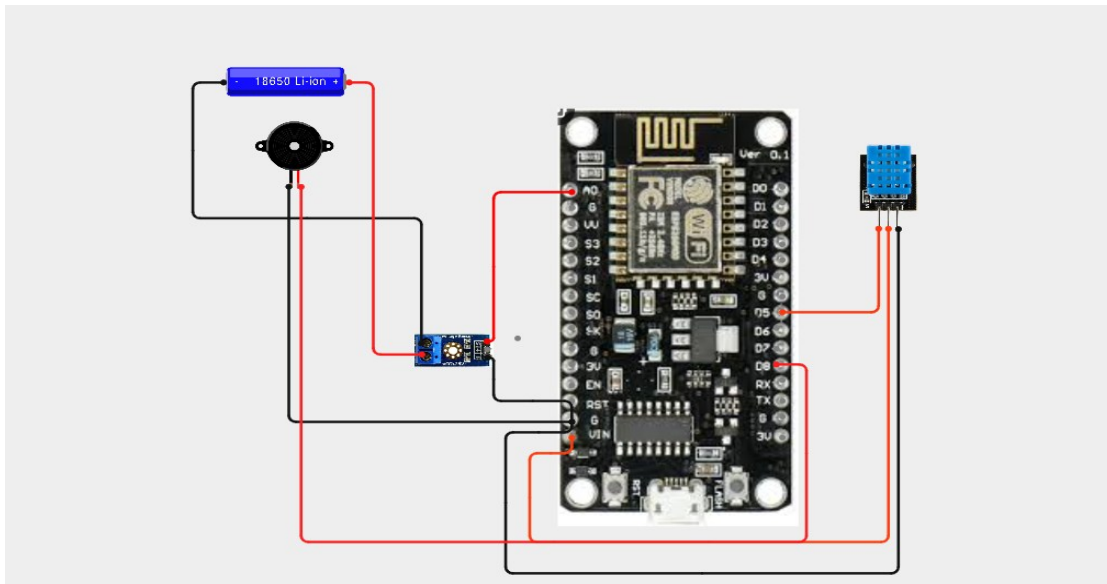
System Architecture:



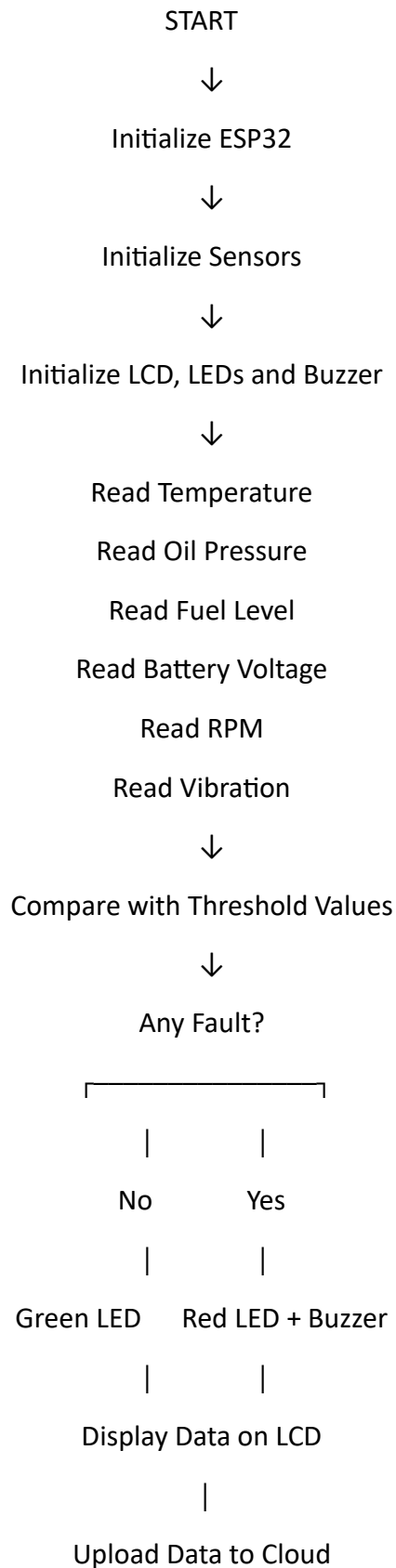
Circuit Table:

Component	Pin	ESP32 Connection	Purpose
DS18B20 Temperature Sensor	DATA	GPIO 4	Engine temperature
Fuel Level Sensor	Analog Out	GPIO 34	Fuel monitoring
Oil Pressure Sensor	Analog Out	GPIO 35	Oil pressure
Voltage Sensor Module	Analog	GPIO 32	Battery voltage
IR RPM Sensor	Digital	GPIO 27	Engine speed
Buzzer	Positive	GPIO 26	Warning alarm
16x2 I2C LCD	SDA/SCL	GPIO 21/22	Display values

Circuit Design:



ALGORITHM:



|

Repeat Continuously

CODING:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>

#define WIFI_SSID "iphone"
#define WIFI_PASS "123456789"
#define IO_USERNAME "Hair_13"
#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

AdafruitIO_WiFi io(
  IO_USERNAME,
  IO_KEY,
  WIFI_SSID,
  WIFI_PASS
);

#define DHT_PIN 2
#define DHT_TYPE DHT11
#define VOLTAGE_PIN A0

#define BUZZER_PIN 15

DHT dht(DHT_PIN, DHT_TYPE);

AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");

AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");

AdafruitIO_Feed *voltageFeed =
  io.feed("voltage");

AdafruitIO_Feed *voltageADCFeed =
  io.feed("voltage-adc");

AdafruitIO_Feed *statusFeed =
```

```
io.feed("status");

AdafruitIO_Feed *warningFeed =
  io.feed("warning");

AdafruitIO_Feed *buzzerFeed =
  io.feed("buzzer");


const float HIGH_TEMPERATURE = 80.0;

const float LOW_VOLTAGE = 200.0;

const float HIGH_VOLTAGE = 250.0;


float voltageCalibration = 1.0;

void setup()
{
  Serial.begin(115200);

  delay(1000);

  // DHT11
  dht.begin();

  // Buzzer
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);

  Serial.println();
  Serial.println("=====");
  Serial.println("DIESEL GENERATOR MONITORING");
  Serial.println("=====");

  Serial.println("Connecting to Adafruit IO...");

  io.connect();

  while (io.status() < AIO_CONNECTED)
  {
    Serial.print(".");
```

```

    delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");

Serial.println("System Ready.");

delay(2000);
}

void loop()
{
    // Keep Adafruit IO connection alive
    io.run();

    float temperature = dht.readTemperature();

    float humidity = dht.readHumidity();
    if (isnan(temperature) || isnan(humidity))
    {
        Serial.println();
        Serial.println("WARNING: DHT11 SENSOR ERROR!");

        digitalWrite(BUZZER_PIN, HIGH);

        buzzerFeed->save("ON");

        statusFeed->save("WARNING");

        warningFeed->save("DHT11 SENSOR ERROR!");

        delay(2000);

        return;
    }

    int voltageADC =
        analogRead(VOLTAGE_PIN);

    float sensorVoltage =
        (voltageADC / 1023.0) * 3.3;

    float generatorVoltage =

```



```
sensorVoltage * voltageCalibration;
```

```
bool warning = false;
```

```
String warningMessage = "NONE";
```

```
if (temperature >= HIGH_TEMPERATURE)
```

```
{
```

```
    warning = true;
```

```
    warningMessage =
```

```
        "HIGH TEMPERATURE!";
```

```
}
```

```
if (generatorVoltage < LOW_VOLTAGE)
```

```
{
```

```
    warning = true;
```

```
    warningMessage =
```

```
        "LOW VOLTAGE!";
```

```
}
```

```
if (generatorVoltage > HIGH_VOLTAGE)
```

```
{
```

```
    warning = true;
```

```
    warningMessage =
```

```
        "HIGH VOLTAGE!";
```

```
}
```

```
if (warning)
```

```
{
```

```
    digitalWrite(BUZZER_PIN, HIGH);
```

```
}
```

```
else
```

```
{
```

```
    digitalWrite(BUZZER_PIN, LOW);
```

```
}
```

```
Serial.println();
```

```
Serial.println("-----");
```

```
if (warning)
```

```
{
  Serial.print("WARNING: ");
  Serial.println(warningMessage);
}
else
{
  Serial.println("SYSTEM NORMAL");
}

Serial.println();

Serial.print("Temperature : ");
Serial.print(temperature, 1);
Serial.println(" C");

Serial.print("Humidity  :");
Serial.print(humidity, 1);
Serial.println(" %");

Serial.print("Voltage ADC : ");
Serial.println(voltageADC);

Serial.print("Voltage  :");
Serial.print(generatorVoltage, 1);
Serial.println(" V");

if (warning)
{
  Serial.println("STATUS   : WARNING");
  Serial.println("BUZZER   : ON");
}
else
{
  Serial.println("STATUS   : NORMAL");
  Serial.println("BUZZER   : OFF");
}
Serial.println("-----");

temperatureFeed->save(temperature);

humidityFeed->save(humidity);

voltageFeed->save(generatorVoltage);
```

```
voltageADCFeed->save(voltageADC);

if (warning)
{
    statusFeed->save("WARNING");
    warningFeed->save(warningMessage);
    buzzerFeed->save("ON");
}
else
{
    statusFeed->save("NORMAL");
    warningFeed->save("NONE");
    buzzerFeed->save("OFF");
}

delay(5000);
}
```

System Working:

The Industrial DG Monitoring System continuously supervises the operating condition of the diesel generator.

- The DS18B20 measures engine temperature.
- The fuel sensor detects remaining diesel level.
- The oil pressure sensor monitors lubrication pressure.
- The voltage sensor checks battery health.
- The IR sensor calculates engine RPM.
- ESP32 compares every value with safety limits.
- LCD displays live generator status.
- If any parameter exceeds its limit, the buzzer immediately alerts the operator.
- Through Wi-Fi, all readings are uploaded to ThingSpeak for remote monitoring and maintenance history.

Bill of Materials:

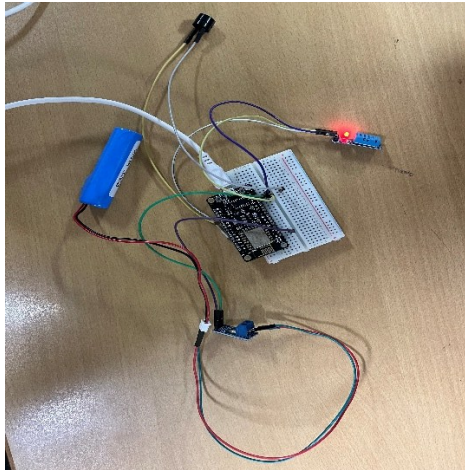
S.No	Component	Quantity	Cost (Rs.)
1	ESP32 Dev Board	1	600
2	DS18B20 Temperature Sensor	1	150
3	Fuel Level Sensor	1	250
4	Oil Pressure Sensor	1	350
5	Voltage Sensor Module	1	120
6	IR RPM Sensor	1	120
7	16×2 I2C LCD	1	180
8	Buzzer & LEDs	1 Set	80
9	Breadboard & Jumper Wires	1	150
	Total		Rs. 2,000

Prototype Implementation:

The prototype is built using an ESP32 controller and multiple sensors mounted on a breadboard.

The sensors are connected to appropriate analog and digital pins. The LCD provides real-time parameter display, while LEDs and buzzer indicate abnormal conditions.

The ESP32 uploads generator health data to a cloud platform, enabling remote monitoring through mobile or web dashboards.



Results & Performance Analysis:

The developed system was tested under different operating conditions.

Parameter	Normal	Warning	Critical
Temperature	70–85°C	86–95°C	>95°C
Fuel Level	>40%	20–40%	<20%
Oil Pressure	30–60 PSI	20–30 PSI	<20 PSI
Battery Voltage	12.4–13.8V	11.8–12.3V	<11.8V
RPM	1450–1550	1300–1450	<1300

Performance:

- Real-time monitoring of 5 generator parameters.
- Immediate fault detection using buzzer alerts. remote monitoring through ThingSpeak cloud.
- Supports preventive maintenance scheduling.
- Low-cost and suitable for industrial applications.

